Modern C++20 Concepts, Ranges & Move Semantics

Constraints, composable views, value categories, ownership, and performance

1. Why these C++20 features fit together

Concepts let templates state requirements directly; ranges let algorithms and views operate on sequences through composable abstractions; move semantics lets types transfer resources rather than unnecessarily duplicate them. Together they can make generic code more expressive while preserving compile-time checking and performance.

```
template<class T>
concept IntegralLike = std::integral<T>;

template<IntegralLike T>
T twice(T value) {
    return value + value;
}
```

Design notes: Include the appropriate headers and compile in C++20 mode, for example with a compiler option such as -std=c++20. Support varies by compiler and standard library version. Prefer standard-library concepts and algorithms over recreating them.

Practice: Inspect a small program's compiler diagnostics when the concept is satisfied and when it is not.

2. Concepts and constraints

A concept is a compile-time predicate describing requirements on types or values. Constraints participate in overload resolution and often produce clearer errors than deeply nested template substitution failures. A concept should describe semantic requirements as well as syntactic expressions where possible.

```
#include <concepts>
#include <string>

template<std::integral T>
T clamp_nonnegative(T value) {
    return value < 0 ? 0 : value;
}

template<class T>
concept HasSize = requires(const T& x) {
    { x.size() } -> std::convertible_to<std::size_t>;
};
```

Design notes: Concepts do not automatically prove every semantic property. A type may satisfy an expression requirement while violating the expected meaning of an operation. Keep constraints focused and avoid over-constraining generic APIs without a real need.

Practice: Write a concept for a type that supports begin() and end(), then compare it with std::ranges::range.

Additional review: Implementation lens: isolate the smallest reproducible example, state the expected result, and test it independently before integrating it into a larger application.

3. Requires-expressions and overload selection

A requires-expression checks whether expressions and type requirements are valid for a candidate type. Constrained overloads can provide a general fallback and a specialized path for types with additional capabilities. Readability matters: complex boolean constraints can be difficult to diagnose.

```
template<class T>
concept Printable = requires(std::ostream& out, const T& value) {
    { out << value } -> std::same_as<std::ostream&>;
};

template<Printable T>
void log_value(const T& value) {
    std::cout << value << '\n';
}
```

Design notes: Remember that the stream insertion expression needs the relevant declarations and headers. Constraint subsumption affects which overload is more specialized; when overload sets become confusing, simplify the constraints and add compile-time tests.

Practice: Try calling log_value with a string, integer, and a custom type with no stream insertion operator.

4. Ranges algorithms

Ranges algorithms accept ranges directly or iterator/sentinel pairs and can constrain their requirements. This removes repetitive begin/end plumbing and can improve diagnostics. Algorithms still have complexity guarantees and preconditions that should be understood.

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> values{9, 2, 7, 1, 5};
    std::ranges::sort(values);
    auto found = std::ranges::find(values, 7);
    if (found != values.end()) std::cout << *found << '\n';
}
```

Design notes: Prefer algorithms over handwritten loops when they communicate intent clearly. Sorting requires a suitable ordering relation; do not assume every user-defined comparator is strict weak ordering. Iterator invalidation rules still apply.

Practice: Replace the vector with a list and explain why ranges::sort cannot be used on it, then choose the appropriate list operation.

5. Views and lazy pipelines

Views describe transformations without necessarily materializing a new container. Pipelines can filter, transform, take, and compose operations. Laziness reduces intermediate allocations but means the source's lifetime and mutation behavior matter.

```
#include <ranges>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> data{1, 2, 3, 4, 5, 6};
    auto pipeline = data
        | std::views::filter([](int n) { return n % 2 == 0; })
        | std::views::transform([](int n) { return n * n; })
        | std::views::take(2);

    for (int value : pipeline) std::cout << value << ' ';
}
```

Design notes: The view may refer to its underlying range rather than owning it. Do not return a view over a local container that is about to be destroyed. Some views cache state or impose restrictions on repeated iteration; know the range category and lifetime.

Practice: Add a second filter and compare the pipeline with an explicitly built vector. Measure rather than assume which is faster.

Additional review: Review lens: consider edge cases, failure behavior, maintainability, and the assumptions the code makes about its input or execution environment.

6. Range categories and lifetime

The ranges library distinguishes input, forward, bidirectional, random-access, and contiguous ranges, with increasingly strong iterator guarantees. Algorithms may require specific categories. A range's iterator validity and ownership are separate questions: a range can be iterable but still dangle if its source dies.

```
auto make_view_bad() {
    std::vector<int> local{1, 2, 3};
    return local | std::views::filter([](int n) { return n > 1; });
    // The returned view can refer to a destroyed vector.
}
```

Design notes: Prefer returning an owning container or use an owning view when appropriate and supported by the design. Be especially cautious with views built from spans, string_views, iterators, or references. Static analysis and sanitizers can catch some lifetime errors but do not prove safety.

Practice: Audit every pipeline in a function for the lifetime of its underlying range.

7. Value categories: lvalues, xvalues, prvalues

C++ expressions have value categories that influence overload resolution and whether a resource can be moved. std::move is a cast that makes an expression an xvalue; it does not itself transfer anything. A move constructor or move assignment operator performs the transfer if selected.

```
#include <string>
#include <utility>

std::string source = "hello";
std::string destination = std::move(source);
// destination now owns the transferred value in typical implementations.
// source remains valid but its value is unspecified.
```

Design notes: After moving from an object, use it only in ways permitted by its type's documented moved-from state, commonly destruction or assignment. Do not rely on a moved-from string being empty unless the contract guarantees it.

Practice: Use overloads taking T&, const T&, and T&& to observe which one is selected for lvalues, const lvalues, and temporaries.

8. Move constructors and resource ownership

A move constructor typically transfers handles or pointers and leaves the source in a valid state. Rule of zero is preferable: compose standard RAII types so the compiler-generated special members are correct. If a type manually owns a resource, its destructor, copy operations, and move operations must be designed consistently.

```
class Buffer {
public:
    explicit Buffer(std::size_t n) : data_(n ? new int[n] : nullptr), size_(n) {}
    ~Buffer() { delete[] data_; }

    Buffer(const Buffer&) = delete;
    Buffer& operator=(const Buffer&) = delete;

    Buffer(Buffer&& other) noexcept
      : data_(std::exchange(other.data_, nullptr)),
        size_(std::exchange(other.size_, 0)) {}

    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete[] data_;
            data_ = std::exchange(other.data_, nullptr);
            size_ = std::exchange(other.size_, 0);
        }
        return *this;
    }
private:
    int* data_;
    std::size_t size_;
};
```

Design notes: This teaching example illustrates ownership transfer; production code should usually prefer std::vector<int> or another RAII member instead of raw new/delete. A noexcept move can help standard containers move elements during reallocation, but only declare it when correct.

Practice: Refactor Buffer to hold std::vector<int>. Identify which special-member functions can then be removed.

Additional review: Optimization lens: measure behavior with representative data and workload; check the relevant official documentation before relying on implementation-specific behavior.

9. Perfect forwarding and universal-reference patterns

A forwarding reference is typically T&& where T is a deduced template parameter. std::forward<T> preserves the argument's value category. This is useful for wrapper functions and factories, but unnecessary forwarding can make interfaces hard to read.

```
#include <utility>

template<class F, class... Args>
decltype(auto) invoke_logged(F&& fn, Args&&... args) {
    // Logging omitted; forwarding preserves value categories.
    return std::forward<F>(fn)(std::forward<Args>(args)...);
}
```

Design notes: std::move always casts to an rvalue-like expression; std::forward conditionally preserves the original category based on template deduction. Do not forward the same object multiple times if the first use may consume it.

Practice: Create a wrapper around a callable that accepts both lvalues and move-only arguments.

10. Performance, API design, and review

Use concepts to make template constraints visible, views to express transformations, and moves where ownership genuinely transfers. Benchmark with representative workloads; abstraction cost depends on optimization, data size, allocation patterns, and compiler. Avoid making code clever at the expense of ownership clarity.

```
template<std::ranges::input_range R>
auto sum_values(const R& range) {
    using T = std::ranges::range_value_t<R>;
    T total{};
    for (const auto& value : range) total += value;
    return total;
}
```

Design notes: The example assumes T supports default construction and +=. A production concept should state those requirements or constrain the function more narrowly. Also note that const R& can affect which ranges are accepted; API design should match the intended range categories.

Practice: Capstone: implement a constrained function that filters and transforms a range, then benchmark materialized and lazy variants with a repeatable input.

Trusted documentation and further reading

- **C++ working draft / reference — Concepts** — https://en.cppreference.com/w/cpp/language/constraints.html
- **C++ working draft / reference — Ranges library** — https://en.cppreference.com/w/cpp/ranges.html
- **C++ working draft / reference — std::move** — https://en.cppreference.com/w/cpp/utility/move.html
- **C++ working draft / reference — Value categories** — https://en.cppreference.com/w/cpp/language/value_category.html
- **C++ working draft / reference — std::ranges::sort** — https://en.cppreference.com/w/cpp/algorithm/ranges/sort.html
- **ISO C++ — C++20 overview** — https://isocpp.org/wiki/faq/cpp20
- **C++ Core Guidelines** — https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

Subject review checklist

- Constraints state real requirements.
- Views do not outlive their underlying ranges.
- Moved-from objects are used only according to their contract.
- Ownership and exception guarantees are clear.
- Benchmarks use representative workloads.